# Generare combinatorială. Backtracking

Infogym • grupa 7–8 • 28 noiembrie 2026

**Andrei Mihai Ciobanu**






**Hai la Olimpiadă!**

# Cuprins


1. Metoda backtracking
2. Elemente combinatoriale
3. Backtracking în plan
4. Alte aplicații
5. Recapitulare și probleme propuse

# Backtracking: definiție

## Definiție

**Backtracking** este o metodă care construiește incremental o soluție, luând câte o decizie la fiecare pas, și parcurge astfel **toate** soluțiile posibile.

**Tipuri de probleme:**

- **Enumerare**: se generează **toate** soluțiile (permutări, combinări etc.)
- **Decizie**: există o soluție? (da / nu)
- **Optimizare**: se caută soluția **cea mai bună**

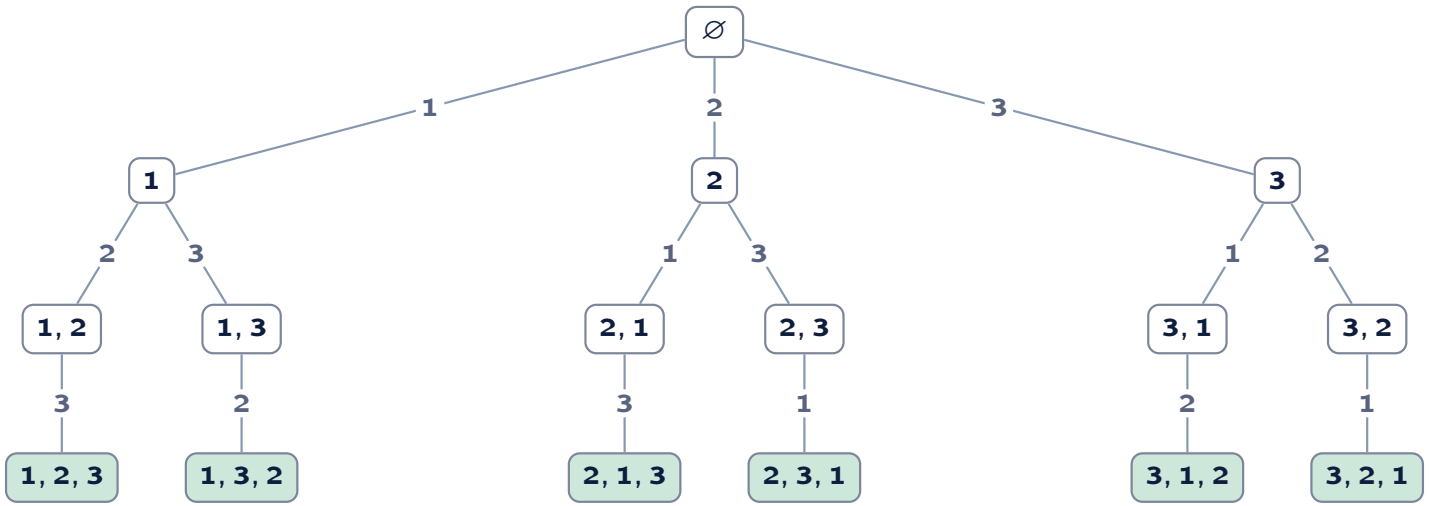
**Ideea centrală:** se construiește un vector soluție $x[1], x[2], \ldots, x[n]$, element cu element. La fiecare pas $k$:

1. se încearcă pe rând **toate valorile posibile** pentru $x[k]$
2. pentru fiecare valoare, se continuă recursiv la pasul $k+1$
3. când toate valorile au fost încercate, se **revine** la pasul $k-1$ (*backtrack*)

**Complexitate:** exponențială, în absența oricărei optimizări.

# Arborele de decizie: permutări ale $\{1, 2, 3\}$

Fiecare nod este starea vectorului soluție, iar fiecare ramură este valoarea aleasă la pasul curent. Frunzele verzi sunt soluții complete.



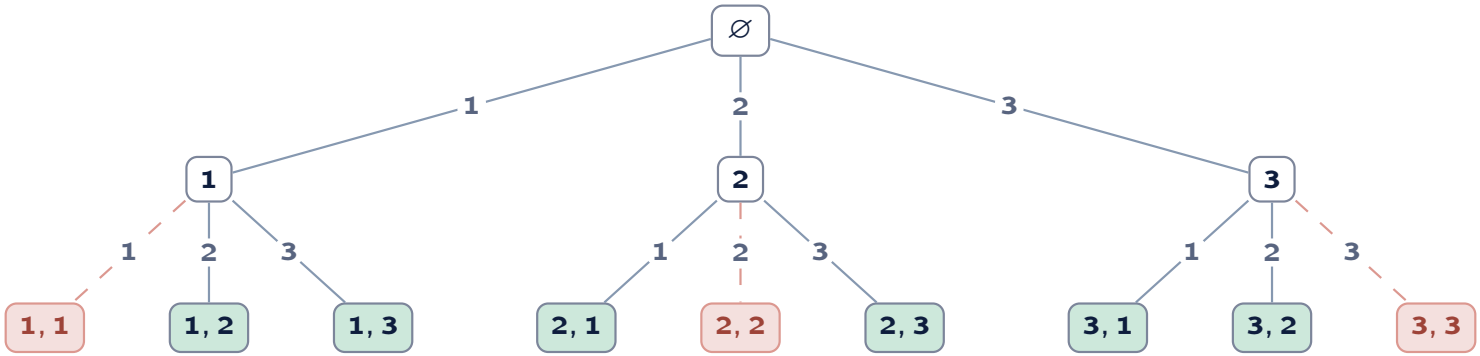

Arborele se explorează **în adâncime** (*Depth First Search*). Pentru $n = 3$: $3! = 6$ frunze, adică 6 permutări distincte.

# Pruning: tăierea ramurilor invalide

## Definiție

**Pruning** înseamnă abandonarea unei ramuri a arborelui de decizie **imediat ce** se constată că nu poate duce la o soluție validă, fără a mai explora subarborele ei.

**Exemplu:** secvențe de lungime 2 din $\{1, 2, 3\}$ cu $x[1] \neq x[2]$. Nodurile roșii sunt tăiate înainte de a continua recursivitatea:




Fără pruning: $3^2 = 9$ frunze. Cu pruning: **6 frunze** (3 ramuri tăiate). La constrângeri mai stricte (de exemplu N regine), reducerea poate fi **dramatică**.

# Schelet general

```
int x[nmax + 1], n;

bool valid(int k) {
    // conditia specifica problemei
    // (x[k] compatibil cu x[1..k-1]?)
    return true;
}

void back(int k) {
    if (k > n) {          // solutie completa
        // afiseaza x[1..n]
        return;
    }
    for (int v = /* valoare_min */; v <= /* valoare_max */; v++) {
        x[k] = v;
        if (valid(k)) {
            back(k + 1);
        }
    }
}
// Apel initial: back(1)
```

**Domeniul valorilor** și **condiția de soluție completă** ($k > n$) sunt specifice fiecărei probleme. Funcția `valid()` poate fi scrisă direct în `back()` dacă este simplă.

# Cuprins


1 Metoda backtracking

2 Elemente combinatoriale

3 Backtracking în plan

4 Alte aplicații

5 Recapitulare și probleme propuse

## Factorial

$n! = 1 \cdot 2 \cdot \ldots \cdot n$ este numărul de moduri de a aranja $n$ obiecte distincte; prin convenție, $0! = 1$.

$$1! = 1, \quad 2! = 2, \quad 3! = 6, \quad 4! = 24, \quad 5! = 120$$

## Permutări ale $\{1, \ldots, n\}$

O **permutare** este o **ordonare** a tuturor celor $n$ elemente. Număr total: $P(n) = n!$.

**Exemplu** ($n = 3$, toate permutările mulțimii $\{1, 2, 3\}$):

| | | | | | | |
|---|---|---|---|---|---|---|
| $x[1]$ | 1 | 1 | 2 | 2 | 3 | 3 |
| $x[2]$ | 2 | 3 | 1 | 3 | 1 | 2 |
| $x[3]$ | 3 | 2 | 3 | 1 | 2 | 1 |

Total: $P(3) = 3! = \mathbf{6}$ permutări

# Permutări: implementare

**Cheia:** `folosit[v]` marchează elementele deja plasate ⇒ fiecare valoare apare o singură dată.

```
int x[nmax + 1];
bool folosit[nmax + 1];
int n;

void back(int k) {
    if (k > n) {
        for (int i = 1; i <= n; i++) {
            cout << x[i] << " ";
        }
        cout << "\n";
        return;
    }
    for (int v = 1; v <= n; v++) {
        if (!folosit[v]) {
            x[k] = v;
            folosit[v] = true;
            back(k + 1);
            folosit[v] = false;    // backtrack
        }
    }
}
// Apel: back(1)
```

## Definiție

Un **aranjament** de $p$ elemente din $n$ ($p \leq n$) este o selecție **ordonată** de $p$ elemente **distincte** din $\{1, \ldots, n\}$.

$$A(n,p) = \frac{n!}{(n-p)!} = n \cdot (n-1) \cdots (n-p+1)$$

**Diferența față de permutări:** se aleg doar $p < n$ elemente, dar **ordinea contează**: $(1,2) \neq (2,1)$.

**Exemplu** ($n = 4$, $p = 2$, din $\{1,2,3,4\}$):

$$(1,2),\ (1,3),\ (1,4),\ (2,1),\ (2,3),\ (2,4)$$
$$(3,1),\ (3,2),\ (3,4),\ (4,1),\ (4,2),\ (4,3)$$

Total: $A(4,2) = 4 \cdot 3 = \frac{4!}{2!} =$ **12** aranjamente

# Aranjamente: implementare

**Identic cu permutările**; singura diferență este oprirea la $k > p$ (au fost alese $p$ elemente), în loc de $k > n$.

```
int x[nmax + 1];
bool folosit[nmax + 1];
int n, p;

void back(int k) {
    if (k > p) {      // am ales p elemente
        for (int i = 1; i <= p; i++) {
            cout << x[i] << " ";
        }
        cout << "\n";
        return;
    }
    for (int v = 1; v <= n; v++) {
        if (!folosit[v]) {
            x[k] = v;
            folosit[v] = true;
            back(k + 1);
            folosit[v] = false;
        }
    }
}
// Apel: back(1)
```

## Definiție

O **combinare** de $p$ elemente din $n$ este o selecție **neordonată** de $p$ elemente distincte din $\{1, \ldots, n\}$.

$$C(n,p) = \binom{n}{p} = \frac{n!}{p!\,(n-p)!} = \frac{A(n,p)}{p!}$$

**Diferența față de aranjamente:** ordinea **nu** contează; $\{1,2\}$ și $\{2,1\}$ sunt **aceeași** combinare. Fiecare combinare corespunde la $p!$ aranjamente $\Rightarrow$ se împarte la $p!$.

**Exemplu** ($n = 4$, $p = 2$, din $\{1,2,3,4\}$):

$$\{1,2\},\ \{1,3\},\ \{1,4\},\ \{2,3\},\ \{2,4\},\ \{3,4\}$$

Total: $C(4,2) = \dfrac{4!}{2! \cdot 2!} = \mathbf{6}$ combinări. Verificare: $C(4,2) = \dfrac{A(4,2)}{2!} = \dfrac{12}{2} = 6$

# Combinări: implementare

**Cheia:** `start = x[k-1] + 1` impune ordinea strict crescătoare $\Rightarrow$ nu apar duplicate. Limita `n - p + k` evită pornirile care nu mai pot fi completate.

```
const int nmax = 10;
int x[nmax + 1];
int n, p;

void back(int k) {
    if (k > p) {
        for (int i = 1; i <= p; i++) {
            cout << x[i] << " ";
        }
        cout << "\n";
        return;
    }
    int start = (k == 1) ? 1 : x[k - 1] + 1;
    for (int v = start; v <= n - p + k; v++) {
        x[k] = v;
        back(k + 1);
    }
}
// Apel: back(1)
```

## Definiție

O **submulțime** a lui $\{1, \ldots, n\}$ este orice selecție de elemente, inclusiv mulțimea vidă $\varnothing$ și întreaga mulțime. În total sunt $2^n$ submulțimi.

**De ce $2^n$?** Fiecare dintre cele $n$ elemente poate fi **inclus sau exclus** independent: $\underbrace{2 \times 2 \times \cdots \times 2}_{n \text{ elemente}} = 2^n$. Spre deosebire de combinări, numărul de elemente alese nu este fixat.

**Exemplu** ($n = 3$, mulțimea $\{1, 2, 3\}$):

| Mască | Submulțime | Mască | Submulțime |
|---|---|---|---|
| 000 | $\varnothing$ | 100 | $\{3\}$ |
| 001 | $\{1\}$ | 101 | $\{1, 3\}$ |
| 010 | $\{2\}$ | 110 | $\{2, 3\}$ |
| 011 | $\{1, 2\}$ | 111 | $\{1, 2, 3\}$ |

Total: $2^3 = 8$ submulțimi

# Submulțimi: alegerea elementului următor

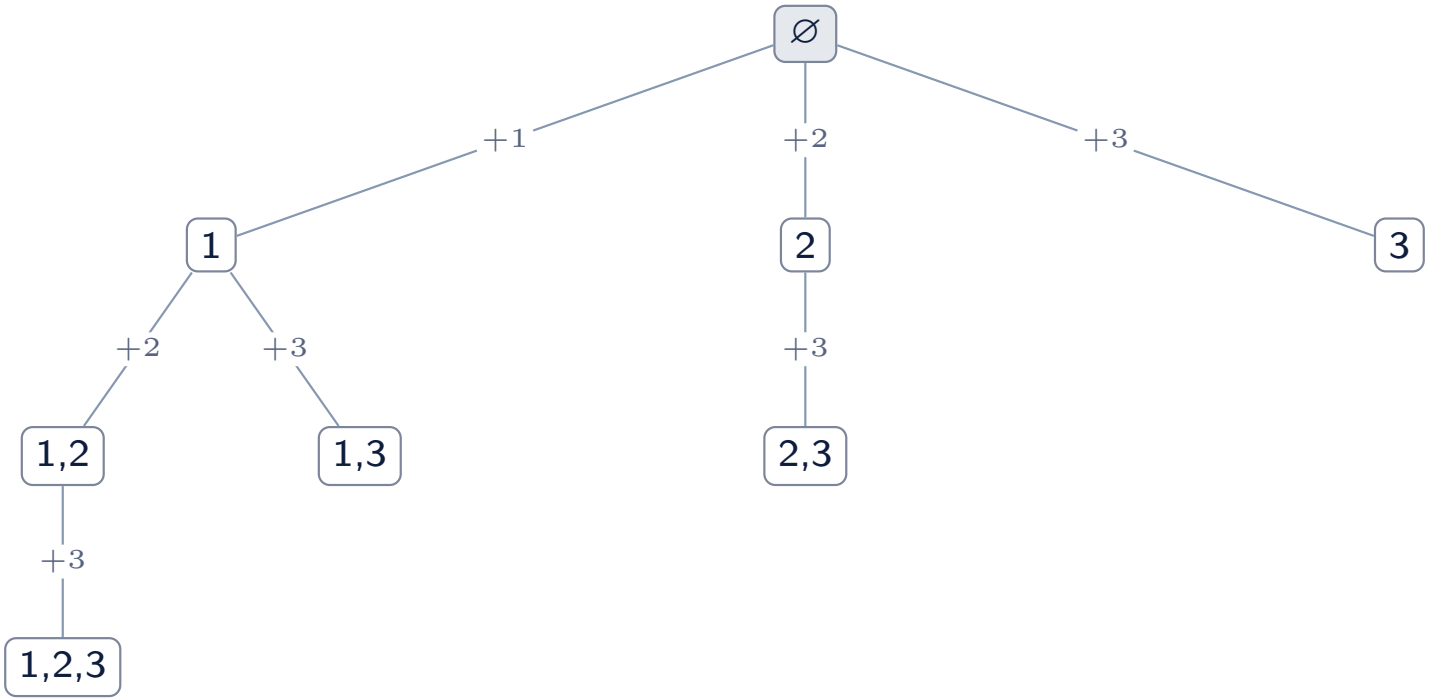
Se afișează submulțimea curentă la fiecare apel, apoi se **extinde** cu elemente mai mari decât ultimul ales. Fiecare apel corespunde unei submulțimi, deci sunt $2^n$ apeluri.

```
int x[nmax + 1], len, n;

void back(int last) {
    for (int i = 1; i <= len; i++) {
        cout << x[i] << " ";
    }
    cout << "\n";
    for (int v = last + 1; v <= n; v++) {
        x[++len] = v;
        back(v);
        len--;
    }
}
// Apel: back(0)
```

# Arborele apelurilor: alegerea elementului următor

Pentru $n = 3$, fiecare nod este o submulțime afișată, iar eticheta unei muchii este elementul adăugat.




Se afișează, în ordine: $\varnothing$, $\{1\}$, $\{1, 2\}$, $\{1, 2, 3\}$, $\{1, 3\}$, $\{2\}$, $\{2, 3\}$, $\{3\}$.

# Submulțimi: include / exclude

La fiecare pas se decide dacă elementul $i$ este **inclus** sau nu.

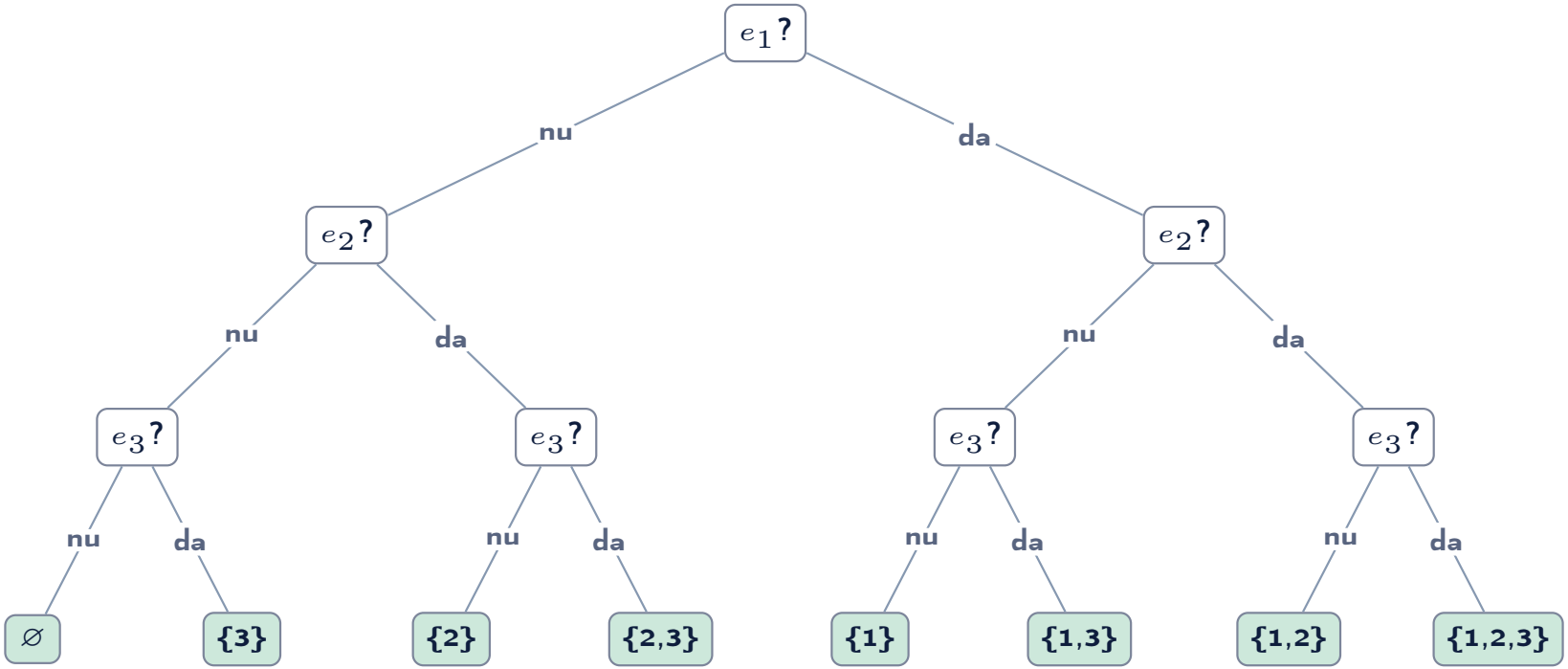
```
int x[nmax + 1], len, n;

void back(int i) {
    if (i > n) {
        for (int j = 1; j <= len; j++) {
            cout << x[j] << " ";
        }
        cout << "\n";
        return;
    }
    back(i + 1);                // nu luam elementul i
    x[++len] = i;
    back(i + 1);                // luam elementul i
    len--;
}
// Apel: back(1)
```

**Observație:** sunt $2^{n+1} - 1$ apeluri față de $2^n$, deoarece nodurile interne nu afișează nimic (decizii goale). La alegerea elementului următor, **fiecare apel afișează o submulțime**.

# Arborele apelurilor: include / exclude

Pentru $n = 3$, fiecare nivel decide soluția pentru un element ($e_i$: inclus sau nu); frunzele sunt cele $2^3 = 8$ submulțimi.



# Submulțimi: implementare cu măști de biți

Fiecare întreg `mask` $\in [0, 2^n)$ codifică o submulțime: bitul $i$ egal cu $1$ înseamnă că elementul $i + 1$ este inclus.

```
int a[nmax + 1], n;

for (int mask = 0; mask < (1 << n); mask++) {
    for (int i = 0; i < n; i++) {
        if (mask & (1 << i)) {
            cout << a[i + 1] << " ";
        }
    }
    cout << "\n";
}
```

- Eficient pentru $n \leq 20$: $2^{20} \approx 10^6$ iterații
- **Combinări cu bitmask:** se reține doar măștile cu exact $p$ biți setați (`__builtin_popcount(mask) == p`). Metoda este mai ineficientă decât backtracking-ul clasic, dar utilă atunci când masca submulțimii există deja

## Definiție

Produsul cartezian $A_1 \times A_2 \times \cdots \times A_n$ este mulțimea tuturor secvențelor $(x_1, \ldots, x_n)$ cu $x_k \in A_k$ pentru fiecare $k$.

$$|A_1 \times \cdots \times A_n| = \prod_{k=1}^{n} |A_k|$$

**Diferența față de permutări:** valorile nu trebuie să fie distincte, iar fiecare poziție are **propriul domeniu**. Dacă $|A_k| = m$ pentru orice $k$, în total sunt $m^n$ secvențe.

**Exemplu:** $A_1 = \{1, 2\}$, $A_2 = \{1, 2, 3\}$. Toate perechile $(x_1, x_2)$:

| | | | | | | |
|---|---|---|---|---|---|---|
| $x_1$ | 1 | 1 | 1 | 2 | 2 | 2 |
| $x_2$ | 1 | 2 | 3 | 1 | 2 | 3 |

Total: $2 \times 3 = \mathbf{6}$ secvențe; perechile $(1, 1)$ și $(2, 2)$ sunt valide, deci repetițiile sunt permise

# Produs cartezian: implementare

**Cheia:** fără `folosit[]`; fiecare poziție $k$ parcurge independent valorile $1 \ldots \texttt{lim}[k]$, deci repetițiile sunt permise.

```
const int nmax = 10;
int x[nmax + 1];
int lim[nmax + 1];
int n;

void back(int k) {
    if (k > n) {
        for (int i = 1; i <= n; i++) {
            cout << x[i] << " ";
        }
        cout << "\n";
        return;
    }
    for (int v = 1; v <= lim[k]; v++) {
        x[k] = v;
        back(k + 1);
    }
}
// Apel: back(1)
```

# Cuprins


1 Metoda backtracking

2 Elemente combinatoriale

3 Backtracking în plan

4 Alte aplicații

5 Recapitulare și probleme propuse

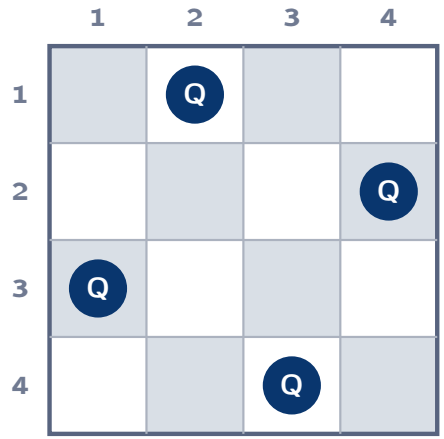
# Problema celor $n$ regine: formulare

## Problemă

Se plasează $n$ regine pe o tablă de șah $n \times n$ astfel încât oricare două regine să nu se atace (pe linie, coloană sau diagonală).

**Modelare:** $x[i]$ este coloana reginei de pe rândul $i$. Deoarece se pune exact o regină pe fiecare rând, conflictele pe linie sunt excluse automat. Rămân de verificat:

- **coloana:** $x[i] \neq x[j]$ pentru $i \neq j$
- **diagonala:** $|x[i] - x[j]| \neq |i - j|$ pentru $i \neq j$




$x = [2, 4, 1, 3]$: soluție pentru $n = 4$

Complexitatea este exponențială, dar pruning-ul reduce semnificativ spațiul de căutare. Pentru $n = 8$ există **92 de soluții**.

# N regine: verificarea poziției

Noua regină se verifică față de toate cele plasate anterior pe rândurile $1 \ldots k-1$.

```
const int nmax = 20;
int x[nmax + 1];
int n;

bool valid(int k) {
    for (int i = 1; i < k; i++) {
        if (x[i] == x[k]) {
            return false; // aceeasi coloana
        }
        if (abs(x[i] - x[k]) == abs(i - k)) {
            return false; // aceeasi diagonala
        }
    }
    return true;
}
```

**Optimizare** $O(1)$**:** în loc de bucla din `valid()`, se mențin tablouri `col[c]`, `diag1[r - c + n]`, `diag2[r + c]` care marchează coloanele și diagonalele ocupate.

# N regine: backtracking

```
void back(int k) {
    if (k > n) {
        for (int i = 1; i <= n; i++) {
            cout << x[i] << " ";
        }
        cout << "\n";
        return;
    }
    for (int col = 1; col <= n; col++) {
        x[k] = col;
        if (valid(k)) {
            back(k + 1);
        }
    }
}
// Apel: back(1)
```

**Pruning:** `valid()` taie ramura imediat. Pentru $n = 8$ se fac $2057$ de apeluri ($15720$ de încercări de plasare), față de $8^8 \approx 1.7 \cdot 10^7$ variante posibile. Rezultatul este de **92 de soluții** pentru $n = 8$.

| $n$ | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|
| soluții | 2 | 10 | 4 | 40 | 92 |
| apeluri back | 17 | 54 | 153 | 552 | 2057 |

# Traseul calului (*Knight's Tour*)

## Problemă

Se plasează un cal de șah pe o tablă $n \times n$ astfel încât să viziteze fiecare celulă **exact o dată** (lanț hamiltonian). Calul se mișcă în L: $(\pm 1, \pm 2)$ sau $(\pm 2, \pm 1)$, deci are cel mult 8 mutări din fiecare celulă.

## Algoritmul lui Warnsdorff (1823): o euristică

La fiecare pas se alege mutarea care duce în celula cu **cele mai puține ieșiri nevizitate**. Backtracking pur: $O(8^{n^2})$. Cu Warnsdorff: $O(n^2)$ în practică.

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1 | 48 | 31 | 50 | 33 | 16 | 63 | 18 |
| 30 | 51 | 46 | 3 | 62 | 19 | 14 | 35 |
| 47 | 2 | 49 | 32 | 15 | 34 | 17 | 64 |
| 52 | 29 | 4 | 45 | 20 | 61 | 36 | 13 |
| 5 | 44 | 25 | 56 | 9 | 40 | 21 | 60 |
| 28 | 53 | 8 | 41 | 24 | 57 | 12 | 37 |
| 43 | 6 | 55 | 26 | 39 | 10 | 59 | 22 |
| 54 | 27 | 42 | 7 | 58 | 23 | 38 | 11 |

**Traseu complet pe tablă $8 \times 8$ (numerele indică ordinea mutărilor)**

# Traseul calului: implementare cu backtracking

```
const int nmax = 8;
int tabla[nmax + 1][nmax + 1];
int dx[] = {-2, -2, -1, -1, 1, 1, 2, 2};
int dy[] = {-1, 1, -2, 2, -2, 2, -1, 1};
int n;

bool valid(int x, int y) {
    return x >= 1 && x <= n && y >= 1 && y <= n
        && tabla[x][y] == 0;
}

void back(int x, int y, int pas) {
    tabla[x][y] = pas;
    if (pas == n * n) {
        // afiseaza tabla[][]
        tabla[x][y] = 0;
        return;
    }
    for (int d = 0; d < 8; d++) {
        int nx = x + dx[d], ny = y + dy[d];
        if (valid(nx, ny)) {
            back(nx, ny, pas + 1);
        }
    }
    tabla[x][y] = 0;      // backtrack
}
// Apel: back(1, 1, 1)
```

# Cuprins


1 Metoda backtracking

2 Elemente combinatoriale

3 Backtracking în plan

4 Alte aplicații

5 Recapitulare și probleme propuse

# Backtracking: când este optim și când nu

- **Permutări, aranjamente, combinări, submulțimi** $\rightarrow$ backtracking **optim** (trebuie generate toate soluțiile)
- **N regine, Sudoku, colorarea grafurilor** $\rightarrow$ backtracking cu pruning **standard**; soluție exponențială inevitabilă
- **Rucsac 0/1:** DP $O(nW)$ când $W$ este mic; backtracking cu Branch & Bound când $W$ este enorm
- **Problema spectacolelor:** backtracking $O(2^n)$, mai bine greedy $O(n \log n)$
- **Traseul calului:** backtracking pur, exponențial; mai bine Warnsdorff $O(n^2)$
- **Lanț / ciclu hamiltonian, SAT, colorare optimală** $\rightarrow$ probleme **NP-complete**: backtracking-ul este abordarea clasică

## Regulă practică

Dacă $n \leq 20$–25, backtracking-ul este viabil. Dacă $n$ este mare, se caută o structură care poate fi exploatată (DP, greedy, Branch & Bound etc.).

## Problema

Se dau $n$ obiecte cu greutățile $w_i$ și profiturile $p_i$, și un rucsac de capacitate $W$. Se maximizează profitul fără a depăși $W$.

**De ce nu DP?** Complexitatea $O(nW)$ nu este acceptabilă la $W \leq 10^9$. Backtracking-ul cu Branch & Bound evită parcurgerea tuturor celor $2^n$ submulțimi.

**Cele 4 optimizări:**

1. **sortare** după $p_i/w_i$ descrescător
2. **bound optimist:** rucsacul fracționar ca limită superioară
3. **inițializare greedy:** un $max_profit$ bun $\Rightarrow$ pruning agresiv
4. **se ia obiectul înainte de a renunța la el**

Dacă $profit + bestSuffix(i, W - weight) \leq max_profit$, ramura se taie (pruning).

# Rucsac 0/1: implementare Branch & Bound

```
// a[i] = {w, p}, sortat dupa p/w descrescator; max_profit initializat greedy
long long weight = 0, profit = 0, max_profit;

long double bestSuffix(int i, long long rem) {
    long double best = 0;
    for (int j = i; j <= n && rem > 0; ++j) {
        long long aux = min(rem, a[j].first);
        best += 1.0L * a[j].second * aux / a[j].first;
        rem -= aux;
    }
    return best;
}

void backtrack(int i) {
    if (i > n) {
        max_profit = max(max_profit, profit);
        return;
    }
    if (profit + bestSuffix(i, W - weight) <= max_profit) {
        return;                              // pruning
    }
    weight += a[i].first;
    profit += a[i].second;
    if (weight <= W) {
        backtrack(i + 1);                    // luam obiectul i
    }
    weight -= a[i].first;
    profit -= a[i].second;
    backtrack(i + 1);                        // nu luam obiectul i
}
```

# `next_permutation`: generare fără recursivitate

**Ideea:** pentru permutarea curentă se găsește *succesoarea* în ordine lexicografică, până când sunt epuizate toate cele $n!$ permutări.

## Algoritmul (3 pași)

1. se găsește cel mai din dreapta $i$ cu $x[i] < x[i+1]$ (sufixul de după $i$ este descrescător)
2. se găsește cel mai din dreapta $j$ cu $x[j] > x[i]$ și se interschimbă $x[i] \leftrightarrow x[j]$
3. se inversează sufixul $x[i+1 \ldots n]$

**Exemplu:** $(1, 3, 2) \to (2, 1, 3)$

- $x[1] = 1 < x[2] = 3$, deci $i = 1$
- cel mai din dreapta $x[j] > 1$ este $x[3] = 2$; interschimbare: $(2, 3, 1)$
- inversarea sufixului $(3, 1)$ dă $(2, 1, 3)$

**Complexitate:** $O(n)$ pe permutare, $O(n \cdot n!)$ în total. Rezultatul este identic cu cel din backtracking, dar fără stivă recursivă.

# next_permutation: implementare

```
bool nextPermutation(int x[], int n) {
    int i = n - 1;
    while (i > 0 && x[i] >= x[i + 1]) {
        i--;
    }
    if (i == 0) {
        return false;            // ultima permutare
    }
    int j = n;
    while (x[j] <= x[i]) {
        j--;
    }
    swap(x[i], x[j]);
    int l = i + 1, r = n;        // inversam sufixul x[i+1..n]
    while (l < r) {
        swap(x[l], x[r]);
        l++;
        r--;
    }
    return true;
}

// Generarea tuturor permutarilor lui {1..n}; initial x[i] = i
do {
    for (int i = 1; i <= n; i++) {
        cout << x[i] << " ";
    }
    cout << "\n";
} while (nextPermutation(x, n));
// sau direct: next_permutation(x + 1, x + n + 1)
```

# Cuprins


1. Metoda backtracking
2. Elemente combinatoriale
3. Backtracking în plan
4. Alte aplicații
5. Recapitulare și probleme propuse

# Recapitulare

| Obiect | Număr | Ce se schimbă în backtracking |
|---|---|---|
| Permutări | $n!$ | vectorul `folosit[]`; oprire la $k > n$ |
| Aranjamente | $\frac{n!}{(n-p)!}$ | ca la permutări; oprire la $k > p$ |
| Combinări | $\binom{n}{p}$ | valori strict crescătoare: de la $x[k-1]+1$ până la $n - p + k$ |
| Submulțimi | $2^n$ | se extinde cu elemente mai mari, sau include / exclude, sau mască de biți |
| Produs cartezian | $\prod_k \lvert A_k \rvert$ | fără `folosit[]`; fiecare poziție are propriul domeniu |

Generarea tuturor soluțiilor costă cel puțin numărul lor înmulțit cu lungimea unei soluții; pruning-ul reduce numărul de noduri explorate, dar nu schimbă complexitatea în cel mai rău caz.

# Termeni importanți

- **Backtracking:** metodă de construire incrementală a unei soluții prin explorarea **tuturor** posibilităților (DFS în arborele de decizie). La fiecare pas se încearcă toate valorile, după care se **revine** (*backtrack*) la pasul anterior
- **Arbore de decizie:** structură în care fiecare nod este o stare parțială a soluției, fiecare ramură este o decizie, iar frunzele sunt soluții complete (sau ramuri tăiate)
- **Pruning (tăierea ramurilor):** abandonarea unui subarbore **imediat ce** se constată că nu poate duce la o soluție validă, fără a-l mai explora. Reduce mult spațiul de căutare la problemele cu constrângeri stricte (N regine, Sudoku)
- **Euristică:** regulă de decizie **aproximativă** care ghidează căutarea spre soluții bune, fără a garanta optimul. Exemplu: algoritmul lui Warnsdorff pentru traseul calului (se alege celula cu cele mai puține ieșiri libere) funcționează foarte bine în practică, dar nu este demonstrat pentru orice $n$

# Probleme propuse: educaționale

| | |
|---|---|
| **All Permutations of 1 to N** | (AlgoCode 1704) |
| **Cartesian Product Generation** | (AlgoCode 1705) |
| **N Queens Board** | (AlgoCode 1364) |
| **Arrangements of k Elements** | (AlgoCode 1373) |
| **Subsets of k Elements** | (AlgoCode 1374) |
| **Nonempty Subsets** | (AlgoCode 1375) |
| **Distinct Word Anagrams** | (AlgoCode 1376) |
| **Balanced Bracket Strings** | (AlgoCode 1412) |

# Probleme propuse: de antrenament (1/2)

| | |
|---|---|
| **Anagrams with Fixed Vowels** | (AlgoCode 1379) |
| **Binary Sequences with Same Digit Counts** | (AlgoCode 1383) |
| **Constrained Digit Generation** | (AlgoCode 1386) |
| **Increasing Sum Decompositions** | (AlgoCode 1403) |
| **Signed Squares Equal N** | (AlgoCode 1404) |
| **Bounded Banknote Payment** | (AlgoCode 1405) |
| **Cube Towers** | (AlgoCode 1406) |

# Probleme propuse: de antrenament (2/2)

| | |
|---|---|
| **Distinct Sum Decompositions** | (AlgoCode 1411) |
| **Fazan Word Chains** | (AlgoCode 1414) |
| **Permutations with Min and Max Fixed** | (AlgoCode 1703) |
| **Blocked Queens** | (AlgoCode 4395) |
| **Digit Difference Numbers** | (AlgoCode 1370) |
| **Special Increasing Sequences** | (AlgoCode 1706) |
| **Reciprocal Sum Sequences** | (AlgoCode 1707) |

# Resurse și bibliografie

1. **pbinfo.ro** — capitolul „Metoda Backtracking”: teorie, pseudocod și probleme clasice (permutări, regine, labirint)
2. **pbinfo.ro** — „Generarea permutărilor / aranjamentelor / combinărilor / submulțimilor”: cod complet pentru fiecare tip de generare
3. **GeeksforGeeks** — „Introduction to Backtracking”: arborele de stări, pruning, N-Queens, Sudoku, cu explicații vizuale
4. **HackerEarth** — „Recursion and Backtracking”: recursivitatea ca mecanism, backtracking-ul ca strategie, exemplu cu labirint
5. **Wikipedia** — „Eight queens puzzle”: 92 de soluții pentru $n = 8$, variante, complexitate
6. **Wikipedia** — „Knight’s tour”: algoritmul lui Warnsdorff, pătrate magice cu traseu de cal